

Relatório - Análise Empírica de Complexidade de Algoritmos

Busca Sequencial, Busca Binária, Selection Sort e Merge Sort

Aluno	Leonardo Xavier Cruz Filho
Matrícula	20250049025
GitHub	Repositório da atividade
Disciplina	Estruturas de Dados Básicos I
Data	Abril de 2026

Trabalho desenvolvido com o objetivo de comparar o comportamento empírico de algoritmos clássicos com suas complexidades assintóticas teóricas, a partir de medições de tempo e análise gráfica.

1. Introdução

A análise de complexidade de algoritmos é uma ferramenta fundamental da computação, pois permite estimar como o tempo de execução de um algoritmo cresce à medida que o tamanho da entrada aumenta. Em geral, essa análise é feita de forma teórica, por meio de notações assintóticas como $O(n)$, $O(\log n)$, $O(n \log n)$ e $O(n^2)$.

Neste trabalho, foi realizada uma análise empírica do tempo de execução de algoritmos clássicos de busca e ordenação, com o objetivo de comparar o comportamento observado experimentalmente com suas complexidades teóricas conhecidas.

Os algoritmos escolhidos foram Busca Sequencial, Busca Binária, Selection Sort e Merge Sort. A proposta consistiu em medir os tempos médios de execução para diferentes tamanhos de entrada, armazenar os resultados e gerar gráficos para verificar se o crescimento empírico acompanhava o comportamento esperado teoricamente.

2. Fundamentação teórica

2.1 Busca Sequencial

A busca sequencial percorre os elementos de um vetor um a um até encontrar o valor desejado ou atingir o final da estrutura. Seu custo cresce linearmente com o tamanho da entrada, sendo, portanto, classificado como $O(n)$.

2.2 Busca Binária

A busca binária opera sobre vetores ordenados. A cada etapa, ela compara o elemento procurado com o elemento central do intervalo e descarta metade do espaço de busca. Por isso, apresenta crescimento logarítmico, sendo classificada como $O(\log n)$.

2.3 Selection Sort

O Selection Sort ordena o vetor selecionando, a cada passo, o menor elemento da parte ainda não ordenada e colocando-o na posição correta. Seu comportamento é quadrático, com complexidade $O(n^2)$.

2.4 Merge Sort

O Merge Sort utiliza a estratégia de divisão e conquista. O vetor é dividido recursivamente em partes menores, que são ordenadas e posteriormente intercaladas. Sua complexidade teórica é $O(n \log n)$, sendo mais eficiente que algoritmos quadráticos para entradas maiores.

3. Metodologia

O projeto foi implementado em C++, com organização modular em arquivos de cabeçalho e arquivos-fonte, facilitando a separação entre algoritmos, geração de dados, medição de tempo e gravação de resultados.

Foram realizados experimentos com tamanhos de entrada crescentes. Para cada tamanho, o tempo de execução de cada algoritmo foi medido diversas vezes, sendo calculada a média para reduzir interferências externas, como variações do sistema

operacional e do hardware. Os pares (n , tempo) foram armazenados em arquivos CSV, permitindo posterior geração de gráficos.

Nos testes de busca, foram utilizados vetores ordenados. A busca sequencial e a busca binária foram aplicadas ao mesmo conjunto de tamanhos de entrada, garantindo comparabilidade entre os algoritmos. Nos testes de ordenação, para cada tamanho n foi gerado um vetor aleatório. Cada algoritmo ordenou uma cópia do mesmo vetor original, garantindo justiça na comparação entre os métodos.

4. Desenvolvimento

O projeto foi dividido em módulos: buscas, ordenações, geração de dados, medição de tempo e utilitários para exportação dos resultados. Essa organização tornou o código mais legível, reutilizável e fácil de expandir, atendendo ao requisito de modularidade do trabalho.

Os resultados de cada algoritmo foram armazenados em arquivos CSV e, em seguida, utilizados por um script em Python para a plotagem dos gráficos comparativos entre os tempos medidos e as curvas teóricas normalizadas.

5. Resultados e análise

Os gráficos obtidos mostraram o comportamento esperado para cada algoritmo. A busca sequencial apresentou crescimento aproximadamente linear, coerente com a complexidade $O(n)$, já que o número de elementos examinados cresce proporcionalmente ao tamanho do vetor.

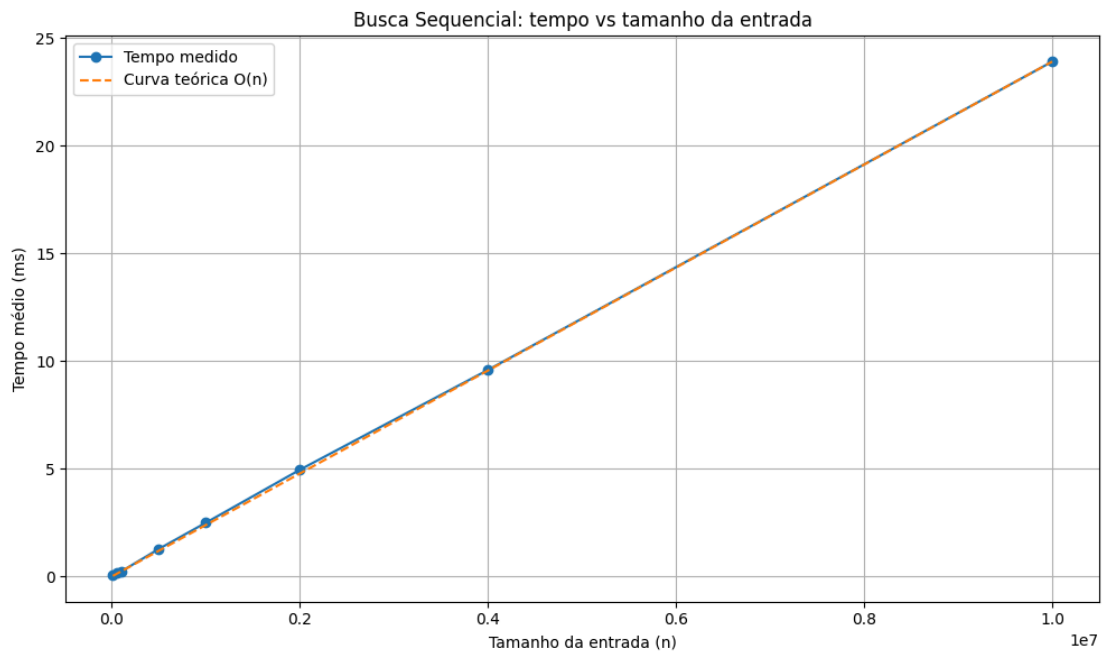
A busca binária apresentou tempos muito baixos, em alguns casos sendo igual a zero na escala utilizada em milissegundos. Esse resultado é compatível com sua complexidade $O(\log n)$, pois o número de comparações cresce muito lentamente mesmo quando o tamanho da entrada aumenta bastante. Assim, embora visualmente os tempos sejam muito pequenos, isso não representa erro, mas sim a alta eficiência do algoritmo.

O Selection Sort apresentou crescimento acelerado do tempo de execução à medida que o tamanho da entrada aumentou, comportamento compatível com a complexidade $O(n^2)$. Esse aumento foi significativamente mais acentuado do que nos demais algoritmos analisados.

O Merge Sort apresentou crescimento inferior ao do Selection Sort e condizente com a complexidade $O(n \log n)$. Nos testes realizados, mostrou-se mais eficiente para entradas maiores, confirmando a vantagem teórica desse algoritmo em relação aos métodos quadráticos.

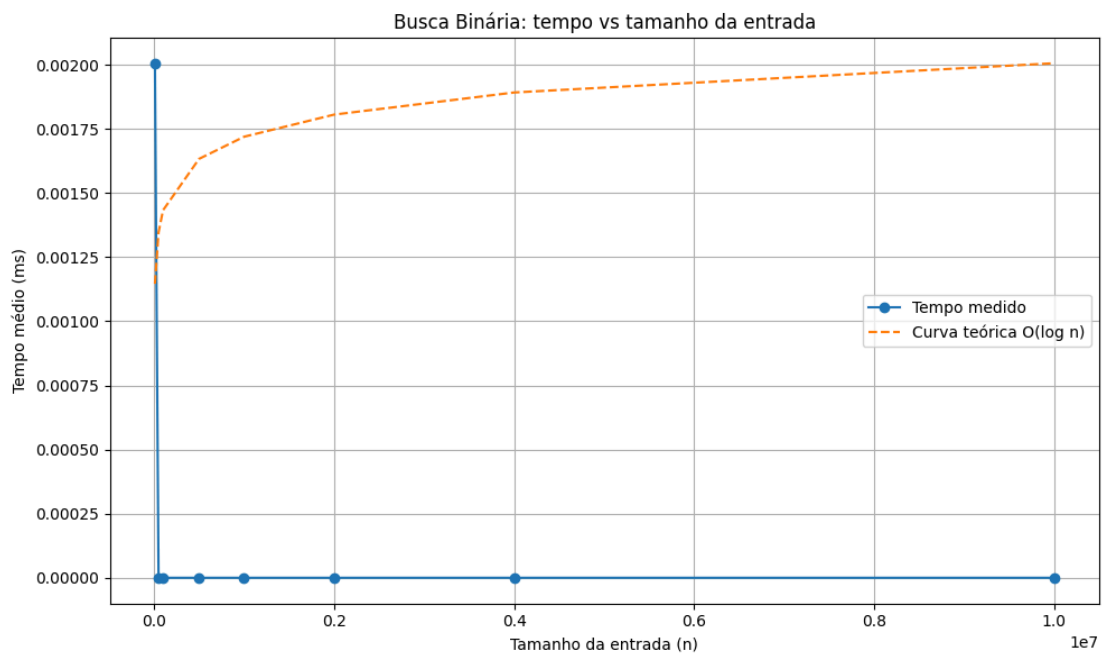
De forma geral, os resultados experimentais ficaram coerentes com a análise teórica de complexidade, mostrando que a observação empírica dos tempos acompanha o comportamento assintótico esperado.

Figura 1 - Busca Sequencial: tempo vs tamanho da entrada



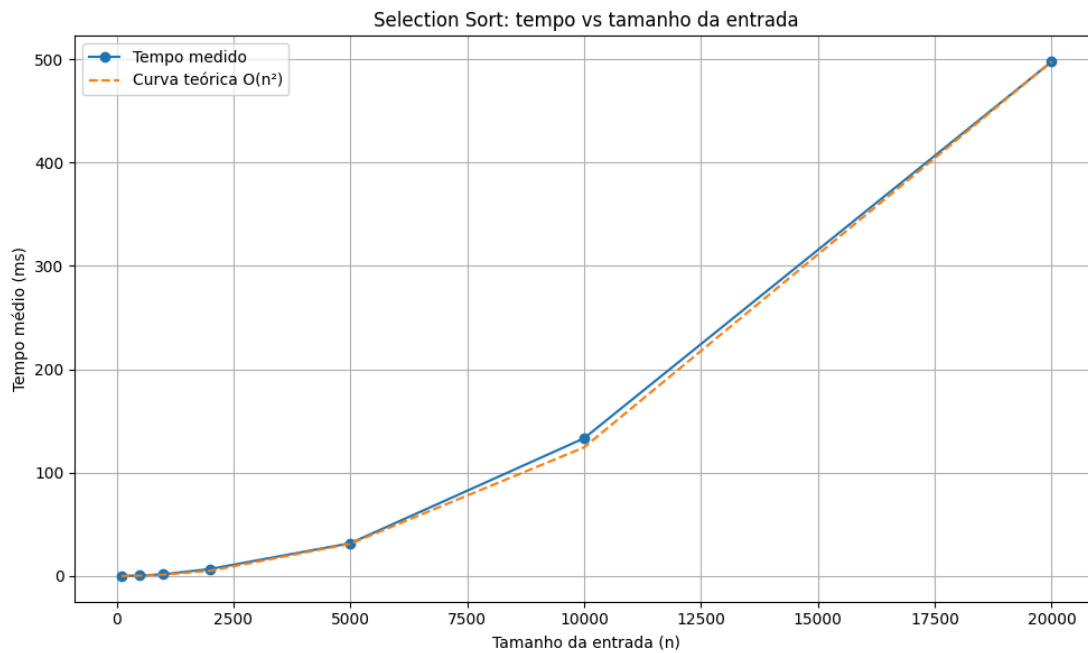
Fonte: elaboração própria.

Figura 2 - Busca Binária: tempo vs tamanho da entrada



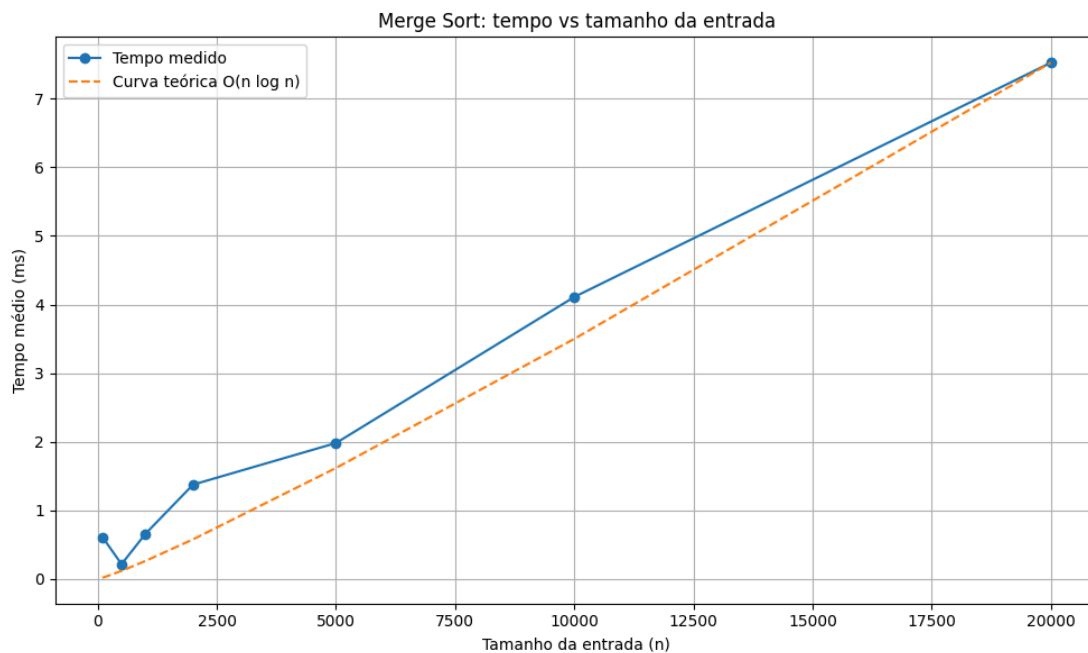
Fonte: elaboração própria.

Figura 3 - Selection Sort: tempo vs tamanho da entrada



Fonte: elaboração própria.

Figura 4 - Merge Sort: tempo vs tamanho da entrada



Fonte: elaboração própria.

6. Conclusão

Este trabalho permitiu comparar, na prática, o comportamento de algoritmos clássicos de busca e ordenação, relacionando medições reais de tempo com a teoria da complexidade assintótica.

Os resultados mostraram que a Busca Sequencial apresentou comportamento linear; a Busca Binária apresentou crescimento logarítmico e tempos muito baixos; o Selection Sort

apresentou crescimento quadrático; e o Merge Sort apresentou crescimento $O(n \log n)$, sendo mais eficiente que o Selection Sort para entradas maiores.

Assim, conclui-se que o comportamento empírico observado está de acordo com a teoria estudada, reforçando a importância da análise de complexidade para a escolha de algoritmos mais adequados a cada problema. Além disso, a comparação entre teoria e prática mostrou-se essencial para compreender como os algoritmos se comportam em situações reais de execução.

7. Referências

Materiais da disciplina de Estruturas de Dados Básicos I.
Enunciado do trabalho prático da Unidade I.